L05 — Insertion and Deletion in Arrays

Unit: 1 | **Chapter:** Fundamentals of Data Structures & Arrays | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Implement insertion at any position in an array
- Implement deletion from any position in an array
- Analyze the time complexity of each operation
- Understand why arrays have O(n) insertion/deletion cost

---

1. Recap: Arrays in Memory

Arrays occupy **contiguous memory**. This is great for access but problematic for insertion/deletion — shifting is required to maintain the contiguous property.

```
Before insertion of 99 at index 2:
Index:  0    1    2    3    4
Value: [10] [20] [30] [40] [50]

After insertion:
Index:  0    1    2    3    4    5
Value: [10] [20] [99] [30] [40] [50]
         ↑    ↑    ↑    elements shifted right
```

---

2. Insertion in an Array

2.1 Insertion at the End — O(1)

The simplest case: no shifting required.

```
def insert_at_end(arr, size, capacity, element):
    if size == capacity:
        raise Exception("Array is full")
    arr[size] = element
    return size + 1  # new size

# Example
arr = [10, 20, 30, 0, 0]  # capacity = 5, size = 3
size = insert_at_end(arr, 3, 5, 40)
# arr = [10, 20, 30, 40, 0], size = 4
```

Time: O(1) | **Space:** O(1)

2.2 Insertion at a Given Position — O(n)

Must shift all elements from position `pos` to `size-1` one step to the right.

```
def insert_at_position(arr, size, capacity, pos, element):
    """
    Insert element at index pos (0-based).
    Valid pos: 0 to size (inclusive).
    """
    if size == capacity:
        raise Exception("Array is full")
    if pos < 0 or pos > size:
        raise Exception("Invalid position")

    # Shift elements right from size-1 down to pos
    for i in range(size, pos, -1):
        arr[i] = arr[i - 1]

    arr[pos] = element
    return size + 1
```

Step-by-step trace for insert 99 at pos = 2 in [10,20,30,40,50]:

Step	Action	Array State
Start	—	[10, 20, 30, 40, 50, _]
i = 5	arr[5] = arr[4] = 50	[10, 20, 30, 40, 50, 50]
i = 4	arr[4] = arr[3] = 40	[10, 20, 30, 40, 40, 50]
i = 3	arr[3] = arr[2] = 30	[10, 20, 30, 30, 40, 50]
i = 2	arr[2] = 99	[10, 20, 99, 30, 40, 50]

Time: O(n) worst case (insertion at index 0 shifts all n elements)
Space: O(1)

2.3 Insertion at Beginning — O(n)

Special case of position insertion with pos = 0. All n elements must shift.

```
def insert_at_beginning(arr, size, capacity, element):
    return insert_at_position(arr, size, capacity, 0, element)
```

---

3. Deletion from an Array

3.1 Deletion from the End — O(1)

Simply reduce the logical size (no actual data erasure needed, though you can zero it).

```
def delete_from_end(arr, size):
    if size == 0:
        raise Exception("Array is empty")
    arr[size - 1] = 0  # Optional: clear the slot
    return size - 1    # new size
```

Time: O(1) | **Space:** O(1)

3.2 Deletion at a Given Position — O(n)

Shift all elements from `pos+1` to `size-1` one step to the left.

```
def delete_at_position(arr, size, pos):
    """
    Delete element at index pos (0-based).
    Valid pos: 0 to size-1.
    """
    if size == 0:
        raise Exception("Array is empty")
    if pos < 0 or pos >= size:
        raise Exception("Invalid position")

    deleted_element = arr[pos]

    # Shift elements left from pos+1 to size-1
    for i in range(pos, size - 1):
        arr[i] = arr[i + 1]

    arr[size - 1] = 0  # Optional: clear last slot
    return size - 1, deleted_element
```

Step-by-step trace for delete at pos = 2 in [10,20,30,40,50]:

Step	Action	Array State
Start	—	[10, 20, 30, 40, 50]
i = 2	arr[2] = arr[3] = 40	[10, 20, 40, 40, 50]
i = 3	arr[3] = arr[4] = 50	[10, 20, 40, 50, 50]
Clear	arr[4] = 0	[10, 20, 40, 50, 0]

Time: O(n) worst case (deletion at index 0 shifts all n-1 elements)
Space: O(1)

3.3 Deletion at Beginning — O(n)

```
def delete_from_beginning(arr, size):
    return delete_at_position(arr, size, 0)
```

---

4. Search Before Deletion

Often we don't know the index — we only know the value. We must first search for it.

```
def delete_by_value(arr, size, target):
    """Delete first occurrence of target."""
    # Step 1: Find the index — O(n)
    pos = -1
    for i in range(size):
        if arr[i] == target:
            pos = i
```

```
            break

    if pos == -1:
        return size, False  # not found

    # Step 2: Delete at found position — O(n)
    size, _ = delete_at_position(arr, size, pos)
    return size, True

# Total: O(n) + O(n) = O(n)
```

---

5. Complexity Summary

Operation	Best Case	Worst Case	Average Case
Insert at end	O(1)	O(1)	O(1)
Insert at position i	O(1)	O(n)	O(n)
Insert at beginning	O(n)	O(n)	O(n)
Delete from end	O(1)	O(1)	O(1)
Delete at position i	O(1)	O(n)	O(n)
Delete from beginning	O(n)	O(n)	O(n)

Why O(n)? In the worst case, inserting at or deleting from index 0 requires shifting all n elements.

---

6. Complete Demo

```
class DynamicArray:
    def __init__(self, capacity):
        self.arr = [0] * capacity
        self.capacity = capacity
        self.size = 0

    def insert(self, pos, val):
        if self.size >= self.capacity:
            raise Exception("Array full")
        for i in range(self.size, pos, -1):
            self.arr[i] = self.arr[i - 1]
        self.arr[pos] = val
        self.size += 1

    def delete(self, pos):
        if pos < 0 or pos >= self.size:
            raise Exception("Invalid index")
        for i in range(pos, self.size - 1):
            self.arr[i] = self.arr[i + 1]
        self.size -= 1

    def display(self):
```

```
        print(self.arr[:self.size])


da = DynamicArray(10)
da.insert(0, 10)   # [10]
da.insert(1, 20)   # [10, 20]
da.insert(2, 30)   # [10, 20, 30]
da.insert(1, 99)   # [10, 99, 20, 30]
da.display()       # → [10, 99, 20, 30]
da.delete(1)       # [10, 20, 30]
da.display()       # → [10, 20, 30]
```

Summary

- **Insertion at end:** O(1) — just append.
- **Insertion at position i:** O(n) — shift elements i to n-1 right by one.
- **Deletion from end:** O(1) — reduce logical size.
- **Deletion at position i:** O(n) — shift elements $i+1$ to n-1 left by one.
- The need to maintain contiguity is why array insertion/deletion is costly — this motivates **linked lists**.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.2 (Springer, 2020)